Анализ

Загребельный Сергей Валерьевич

14 февраля 2026 г.

Содержание

1 Четкость постановки задачи **2**

2 Математическая модель **2**

3 Алгоритм решения **2**

4 Тестирование и верификация **3**

5 Общие характеристики алгоритма **4**

6 Выявленные проблемы и способы их решения **4**

7 Вывод **4**

1 Четкость постановки задачи

Постановка задачи является чёткой и однозначной. Входным параметром является натуральное число N, обозначающее суммарное количество голов в стае драконов. Также задано ограничение, что один дракон может иметь не более 7 голов. Требуется определить максимальную силу стаи, которая вычисляется как произведение количества голов каждого дракона, при условии, что сумма голов всех драконов равна N.
Таким образом, входные данные, ограничения и требуемый результат сформулированы ясно, что позволяет однозначно определить цель решения задачи и выбрать соответствующий алгоритм.

2 Математическая модель

Математически задача сводится к следующей задаче оптимизации:
необходимо представить число N в виде суммы натуральных чисел:

$$N = x_1 + x_2 + \cdots + x_m,$$

где

$$1 \leqslant x_i \leqslant 7,$$

и максимизировать произведение:

$$P = x_1 \cdot x_2 \cdot \cdots \cdot x_m.$$

Это задача максимизации произведения при фиксированной сумме и ограничениях на значения слагаемых. Такая задача относится к классу задач оптимального разбиения числа и эффективно решается методом динамического программирования.

3 Алгоритм решения

Для решения задачи был выбран метод динамического программирования, который позволяет эффективно находить оптимальное решение путём использования ранее вычисленных результатов.
Был введён массив:

$$dp[s] = \text{максимальное произведение для суммы } s.$$

Алгоритм работает следующим образом:

1. Инициализируется массив dp размером $N + 1$.
2. Устанавливается базовое значение:
$$dp[0] = 1.$$
3. Для каждой суммы s от 1 до N перебираются все возможные значения количества голов одного дракона k от 1 до 7.
4. Если $s - k \geqslant 0$, вычисляется кандидат:
$$candidate = dp[s - k] \cdot k.$$
5. Выбирается максимальное значение среди всех кандидатов:
$$dp[s] = \max(dp[s], candidate).$$
6. После заполнения массива ответом является значение $dp[N]$.

Данный алгоритм гарантирует нахождение оптимального решения, так как перебирает все возможные допустимые варианты разбиения числа с использованием принципа оптимальности.

4 Тестирование и верификация

Были рассчитаны и добавлены тесты для разных чисел.

```
test_cases = [
        /* (вход, ожидаемый результат) */
        (1, 1),
        (2, 2),
        (3, 3),
        (4, 4),
        (5, 6),   /* 3+2 => 6      */
        (6, 9),   /* 3+3 => 9      */
        (7, 12),  /* 3+4 => 12     */
        (8, 18),  /* 3+3+2 => 18   */
        (9, 27),  /* 3+3+3 => 27   */
        (10, 36), /* 3+3+4 => 36   */
        (11, 54), /* 3+3+3+2 => 54 */
        (12, 81), /* 3+3+3+3 => 81 */
        (13, 108),
        (14, 162),
        (15, 243),
        (20, 1458),
        (30, 59049),
        (50, 86093442),
        (99, 5559060566555523),
        ]
```

5 Общие характеристики алгоритма

Разработанный алгоритм обладает следующими характеристиками:

- Корректность — гарантирует нахождение максимального произведения.
- Эффективность — временная сложность:

$$O(N)$$

- Низкое потребление памяти:

$$O(N)$$

- Универсальность — применим для любых значений N в пределах заданных ограничений.
- Простота реализации и понимания.

6 Выявленные проблемы и способы их решения

В ходе выполнения задачи были выявлены следующие потенциальные проблемы:

- **Проблема:** полный перебор всех возможных разбиений имеет экспоненциальную сложность.

  Решение: использование динамического программирования позволяет сократить сложность до линейной.

- **Проблема:** возможность повторного вычисления одинаковых подзадач.

  Решение: сохранение промежуточных результатов в массиве dp исключает повторные вычисления.

- **Проблема:** необходимость учитывать ограничение (не более 7 голов у дракона).

  Решение: ограничение учитывается в цикле перебора значений от 1 до 7.

- **Проблема:** корректная работа для всех допустимых значений N.

  Решение: алгоритм протестирован на малых, средних и граничных значениях.

7 Вывод

В результате выполнения кейс-задачи был разработан эффективный алгоритм на основе динамического программирования, позволяющий находить максимальную силу стаи драконов при заданном количестве голов.

Алгоритм является корректным, эффективным и полностью удовлетворяет требованиям задачи. Использование динамического программирования позволило значительно сократить время вычислений и обеспечить оптимальность решения.